UNIT II

Stacks: Definition – operations - applications of stack.
Queues: Definition -operations - Priority queues - De queues – Applications of queue.
Linked List: Singly Linked List, Doubly Linked List, Circular Linked List, linked stacks, Linked queues, Applications of Linked List.

2 MARKS

1. What are all the Common data structures?

- Stacks
- Queues
- Lists
- Trees
- Graphs
- Tables

2. What are the two basic types of Data structures?

1. Primitive Data structure

   Eg., int,char,float

2. Non Primitive Data Structure

   i. Linear Data structure

      Eg., Lists Stacks Queues

   ii. Non linear Data structure

      Eg., Trees Graphs

3. What are the four basic Operations of Data structures?

1. Traversing
2. Searching
3. Inserting
4. Deleting

4. What is a stack? (April 2012, April 2013)

- A ***stack*** is an ordered collection of items where new items may be ***inserted*** or ***deleted*** only at one end, called the ***top*** of the stack.
- A stack is a data structure that keeps objects in Last- In-First-Out (***LIFO***) order,
- Objects are added to the top of the stack
- Only the top of the stack can be accessed.

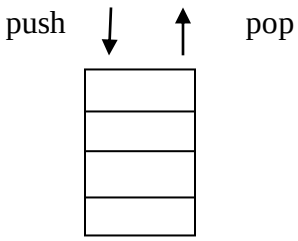
5. What are the basic operations of stack?

- Push
- Pop
- Top

Push: Push is performed by inserting at the top of stack.

Pop: pop deletes the most recently inserted element.

Top: top operation examines the element at the top of the stack and returns its value.



6. Define abstract data type(ADT) and list its advantages? (April 2013)

ADT is a set of operations.

An abstract data type is a data declaration packaged together with the operations that are meaning full on the data type.

Abstract Data Type

- Declaration of data
- Declaration of operations.

Advantages:

- Easy to debug small routines than large ones.
- Easy for several people to work on a modular program simultaneously.
- A modular program places certain dependencies in only one routine, making changes easier.

7. What are the operations of ADT?

Union, Intersection, size, complement and find are the various operations of ADT.

8. State the different ways of representing expressions?

The different ways of representing expressions are

- Infix Notation
- Prefix Notation
- Postfix Notation

9. State the rules to be followed during infix to postfix conversions? (Nov 2012)

- Fully parenthesize the expression starting from left to right. During parenthesizing, the operators having higher precedence are first parenthesized

- Move the operators one by one to their right, such that each operator replaces their corresponding right parenthesis
- The part of the expression, which has been converted into postfix is to be treated as single operand

10. Write the various applications of the stack?

- Towers of Hanoi
- Reversing a string
- Balanced parenthesis
- Recursion using stack
- Evaluation of arithmetic expressions
- Infix to postfix conversion

11. Write the algorithm for balancing symbols?

1. Make an empty stack.
2. Read characters until end of file.
3. If the character is an opening symbol, then push it onto the stack.
4. If it is a closing symbol Then

    If the stack is empty

    Report an error

    Otherwise pop the stack

5.If the symbol popped is not the corresponding openinig symbol

    Then

    Report an error

6. If the stack is not empty at the end of file

    Then

    Report an error

12. Give the Features of balancing symbols?

- It is clearly linear.
- Makes only one pass through the input.
- It is online and quite fast.
- It must be decided what to do when an error is reported.

13. Define Infix notation?

- The operator symbol is placed in between its two operands. This is called ***infix notation***.

    *Example: A + B , E * F*

- Parentheses can be used to group the operations.

    *Example: (A + B) * C*

14. Define Prefix notation? (April 2011)

- Polish notation refers to the notation in which the operator symbol is placed before its two operands. This is called ***prefix notation***.

  *Example: +AB, *EF*

- The fundamental property of polish notation is that the order in which the operations are to be performed is completely determined by the positions of the operators and operands in the expression.
- Accordingly, one never needs parentheses when writing expressions in Polish notation.

15. Define Postfix notation? (April 2011)

- ***Reverse Polish Notation*** refers to the analogous notation in which the operator symbol is placed after its two operands. This is called ***postfix notation***.

  *Example: AB+, EF**

- Here also the parentheses are not needed to determine the order of the operations.

16. Convert the given infix to postfix and prefix?

(j*k)+(x+y)

postfix : jk* xy++

prefix : +*jk+xy

17. Convert into postfix and evaluate the following expression.?

(a+b*c)/d ; a=2 b=4 c=6 d=2

Post fix:

abc*+d/

Evaluation:

2 4 6 * +2/ =13

18. Write the features of representing calls in a stack?

- When a function is called the register values and return address are saved.
- After a function has been executed the register values are resumed on returning to the calling statement.
- The stack is used for resuming the register values and for returning to the calling statement.
- The stack overflow leads to fatal error causing loss of program and data.
- Recursive call at the last line of the program is called tail recursion and also
- leads to error.

19. What is the difference between storing data on the heap vs. on the stack?

The stack is smaller, but quicker for creating variables, while the heap is limited in size only by how much memory can be allocated. Stack would include most compile time variables, while heap would include anything created with malloc or new.

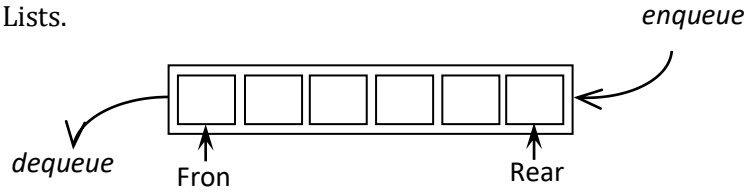
20. What is the data structures used to perform recursion?

Stack. Because of its LIFO (Last In First Out) property it remembers its 'caller' so knows whom to return when the function has to return. Recursion makes use of system stack for storing the return addresses of the function calls.

Every recursive function has its equivalent iterative function. Even when such equivalent iterative procedures are written, explicit stack is to be used.

21. Define a Queue? (April 2012)

Queue is an ordered collection of elements in which insertions are restricted to one end called the rear end and deletions are restricted to other end called the front end. Queues are also referred as First-In-First-Out (FIFO) Lists.



e.g: Ticket counter.

22. List the operations of queue? (April 2011, April 2013)

Two operations

- Enqueue-inserts an element at the end of the list called the rear.
- Dequeue-delets and returns the element at the start of the list called as the front.

23. How the enqueue and dequeue operations are performed in queue?

To enqueue an element X:

increment size and rear

set queue[rear]=x

To dequeue an element

set the return value to queue[front].

Decrement size

increment front

24. Write the routines for enqueue operation in queue?

```
Void
Enqueue(Element type X,Queue Q)
{
If(IsFull(Q))
Error("Full queue");
Else
{
Q->Size++;
Q->Rear=Succ(Q->Rear,Q);
Q->Array[Q->Rear]=X;
}}
```

25. Write the routines for dequeue operation in queue?

```
Void Dequeue(Queue Q)
{
If(IsEmpty(Q))
Error("Empty Queue");
Else
Q->front++;
}
```

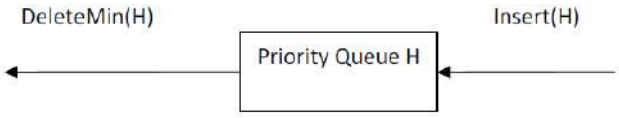
26. What is the difference between a queue and a stack?

Queue	Stack
Queue is typically FIFO	Stack is LIFO
Elements get inserted at one end of a queue and retrieved from the other	Insertion and removal operations for a stack are done at the same end.

27. What are priority queues?

A priority queue is a collection of elements such that each element has been assigned a priority.

- Insert-inserts an element at the end of the list called the rear.
- DeleteMin-Finds, returns and removes the minimum element in the priority Queue.



28. What are the types of priority queues?

- Ascending Priority Queue
- Descending Priority Queue

39. What is an ascending priority queue?

It is a collection of items into which items can be inserted arbitrarily and from which the smallest item can be removed.

30. What is an descending priority queue?

It is a collection of items into which items can be inserted arbitrarily and from which the largest item can be removed.

31. Give the applications of priority queues?

There are three applications of priority queues

1. External sorting.
2. Greedy algorithm implementation.
3. Discrete even simulation.
4. Operating systems.

32. How do you test for an empty queue?

To test for an empty queue, we have to check whether REAR=HEAD where REAR is a pointer pointing to the last node in a queue and HEAD is a pointer that points to the dummy header. In the case of array implementation of queue, the condition to be checked for an empty queue is REAR=max-1, front=rear+1.

33. Define a Dequeue?

Dequeue is a list that combines the properties of a stack and queue. It is linear list in which insertions and deletions are made to or from either end of the structure.

34. Give the minimum number of queues needed to implement the priority queue?

Two. One queue is used for actual storing of data and another for storing priorities.

35. List the applications of Queues?

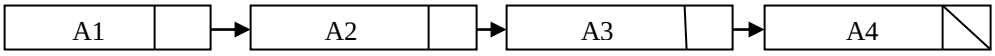
- Checking strings of a language
- Queuing theory simulation
- Input / Output buffers
- Graph searching
- Jobs sent to a printer
- Line in a ticket counter

36. How can you overcome the limitations of arrays?

Limitations of arrays can be solved by using the linked list.

37. Define Linked Lists? Or What is Automatic list management?(April 2012)

Linked list consists of a series of structures, which are not necessarily adjacent in memory. Each structure contains the element and a pointer to a structure containing its successor. We call this theNext Pointer. The last cell'sNext pointer points to NULL.



38. Define HEAD pointer and NULL pointer?

HEAD-It contains the address of the first node in that list.

NULL - It indicates the end of the list structure.

49. What is the need for the header?

Header of the linked list is the first element in the list and it points to the first data element of the list, i.e it contains the address of the first element in the list. If needed, it may store the number of elements in the list.

40. What is a node?

The data element of a linked list is called a node.

41. What does node consist of?

Node consists of two fields: data field to store the element and link field to store the address of the next node.

42. List three examples that uses linked list?

- Polynomial ADT
- Radix sort
- Multi lists

43. State the difference between arrays and linked lists ?

Arrays	Linked Lists
Size of an array is fixed	Size of a list is variable
It is necessary to specify the number of elements during declaration.	It is not necessary to specify the number of elements during declaration
Insertions and deletions are somewhat difficult	Insertions and deletions are carried out easily
It occupies less memory than a linked list for the same number of elements	It occupies more memory

.

44. List the basic operations carried out in a linked list?

The basic operations carried out in a linked list include:

- Creation of a list
- Insertion of a node
- Deletion of a node
- Modification of a node
- Traversal of the list

45. List out the different ways to implement the list?

1. Array Based Implementation
2. Linked list Implementation
    i. Singly linked list
    ii. Doubly linked list
    iii. Cursor based linked list

46. List out the advantages of using a linked list?

- It is not necessary to specify the number of elements in a linked list during its declaration
- Linked list can grow and shrink in size depending upon the insertion and deletion that occurs in the list
- Insertions and deletions at any place in a list can be handled easily and efficiently
- A linked list does not waste any memory space

47. List out the disadvantages of using a linked list?

- Searching a particular element in a list is difficult and time consuming
- A linked list will use more storage space than an array to store the same number of elements

48. State the different types of linked lists?

The different types of linked list include

i. Singly linked list
ii. Doubly linked list
iii. Cursor based linked list

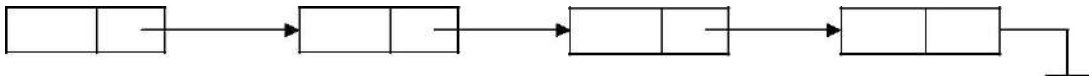
49. What are the advantages of Linked List over arrays?

1. In Linked List implementation Memory location need not be necessarily contiguous.
2. Insertion and deletions are easier and needs only one pointer assignment.
3. A small amount of memory is been wasted for storing a pointer, which is been associated with each node.

50. What is a singly Linked List? (April 2013)

A singly linked list is a collection of nodes each node is a structure it consisting of an element and a pointer to a structure containing its successor, which is called a next pointer.

The last cell's next pointer points to NULL specified by zero.



51. List the operations of single linked list?

- MakeEmpty
- IsEmpty
- IsLast
- Find
- Delete
- FindPrevious
- Insert
- Deletelist
- Header
- First
- Advance
- Retrieve

52. List out the advantages and disadvantages of singly linked list?

Advantages:

- Easy insertion and deletion.
- Less time consumption.

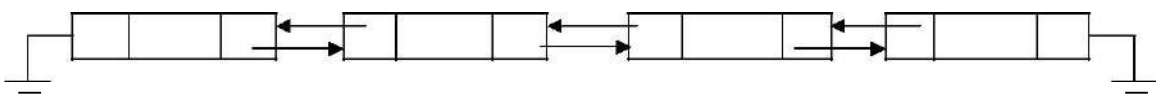
Disadvantages:

- Data not present in a linear manner.
- Insertion and deletion from the front of the list is difficult without the use of header node.

53. What is a doubly linked lists? (Nov 2012)

Doubly linked list is a collection of nodes where each node is a structure containing the following fields

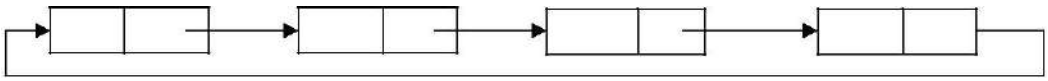
1. Pointer to the previous node.
2. Data.
3. Pointer to the next node.



54. What is a circularly linked lists?

Circularly linked list is a collection of nodes , where each node is a structure containing the element and a pointer to a structure containing its successor.

The pointer field of the last node points to the address of the first node. Thus the linked list becomes circular.



55. Compare Single linked list and double linked list?

Singly	Doubly
It is a collection of nodes and each node is having one data field and next link field	It is a collection of nodes and each node is having one data field one previous link field and one next link field
The elements can be accessed using next link	The elements can be accessed using both previous link as well as next link
No extra field is required hence, Node takes less memory in SLL.	One field is required to store previous Link Hence, node takes memory in DLL.
Less efficient access to elements	More efficient access to elements.

56. Write the difference between doubly and circularly linked list?

Doubly	Circularly
If the pointer to next node is Null, it specifies the last node.	There is no first and last node.
Last node's next field is always Null	Last nodes next field points to the address of the first node.
Every node has three fields one is Pointer to the pervious node, next Is data and the third is pointer to the next node.	It can be single linked list and double linked list

57. State Linked list implementation of stack?

- Push operation is performed by inserting an element at the front of the list.
- Pop operation is performed by deleting at the front of the list.
- Top operation returns the element at the front of the list.

58. State Linked List implementation of Queue?

Enqueue operation is performed at the end of the list.

Dequeue operation is performed at the front of the list.

59. What are the conditions that could be followed in a linked list implementations of queue?

Condition	Situation
REAR==HEAD	EMPTY QUEUE
REAR==LINK (HEAD)	ONE ENTRY QUEUE
NO FREE SPACE TO INSERT	FULL QUEUE

60. List out the applications of a linked list?

Some of the important applications of linked lists are

- manipulation of polynomials,
- sparse matrices,
- stacks and queues.

61. Whether Linked List is linear or Non-linear data structure?

According to Access strategies Linked list is a linear one. According to Storage Linked List is a Non-linear one.

62. State the disadvantages of linked list implementation of stack?

1. Calls to malloc and free functions are expensive.
2. Using pointers is expensive.

63. List down the rules to evaluate the postfix expression. (Nov 2012)

• Initialize an empty stack

• While token remain in the input stream

– Read next token

– If token is a number, push it into the stack

– Else, if token is an operator, pop top two tokens off the stack, apply the operator, and push the answer back into the stack

• Pop the answer off the stack

64. What is length and height Stack(Nov 2012)

Length and Height of stack is Top Pointer Position.

65. What are the prefix and postfix forms of the expression?(April 2011)

Infix => (a + b) * (c – d) / (e + f)

Prefix => / * + a b - c d + e f

Infix => A / B – C + D * E – A * C

Postfix => A B / C – D E * + A C * -

11 MARKS

1. Define Stack? Explain the various operations in Stacks? (April 2012, April 2013)

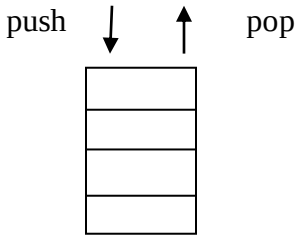
STACK

- A ***stack*** is an ordered collection of items where new items may be ***inserted*** or ***deleted*** only at one end, called the ***top*** of the stack.
- A stack is a data structure that keeps objects in Last- In-First-Out (***LIFO***) order,
- Objects are added to the top of the stack
- Only the top of the stack can be accessed.

Stacks have some useful terminology associated with them:

- **Push** To add an element to the stack
- **Pop** To remove an element from the stock
- **Peek** To look at elements in the stack without removing them
- **LIFO** Refers to the last in, first out behavior of the stack
- **FILO** Equivalent to LIFO

Stack (data structure)



Simple representation of a stack:

Given a stack S=(a[1],a[2],.....a[n]) then we say that a1 is the bottom most element and element a[i]) is on top of element a[i-1], 1<i<=n.

Implementation of stack:

1. Array (static memory)
2. linked list (dynamic memory)

Operations of stack is

I. PUSH operations

II. POP operations

III. PEEK operations

The Stack ADT

A stack S is an abstract data type (ADT) supporting the following three methods:

push(n) : Inserts the item ***n*** at the top of stack

pop() : Removes the top element from the stack and returns that top element. An error occurs if the stack is empty.

peek() :Returns the top element and an error occurs if the stack is empty.

I. PUSH operation - Adding an element into a stack

Adding element into the TOP of the stack is called PUSH operation.

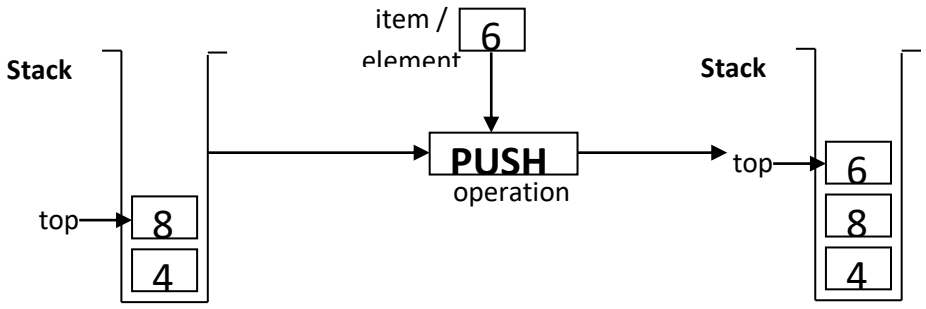
Check conditions :

TOP = N , then **STACK FULL**

where N is maximum size of the stack.

PUSH algorithm- Adding an element into stack

```
procedure add(item : items);
{add item to the global stack stack ;   top is the current top of stack
and n is its maximum size}
begin
  if top = n then stackfull;
  top := top+1;
  stack(top) := item;
end: {of add}
```



Implementation in C using array:

```
/* here, the variables stack, top and size are global variables */
 void push (int item)
{
        if (top == size-1)
                printf("Stack is Overflow");
        else
        {
```

```
            top = top + 1;
            stack[top] = item;
        }}
```

II. POP operation - Deleting an element from a stack.

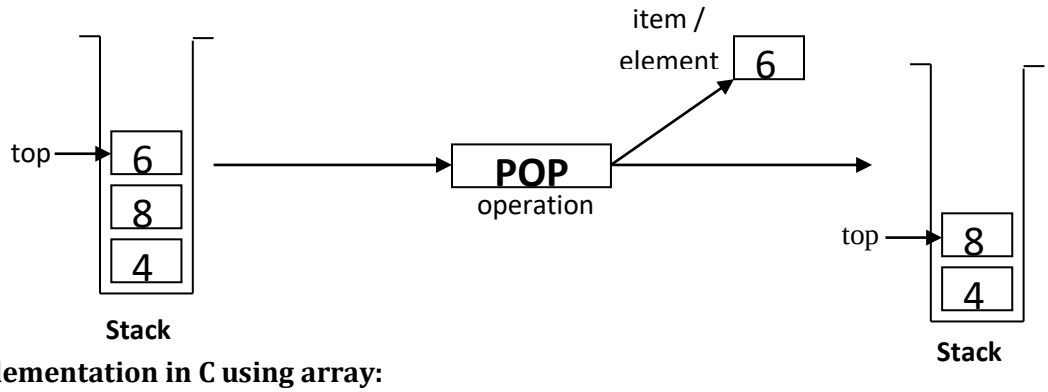
Deleting or Removing element from the TOP of the stack is called POP operations.

Check Condition:

TOP = 0**, then** STACK EMPTY

POP algorithm- Deleting an element from a stack

```
    procedure delete(var item : items);
    {remove top element from the stack stack and put it in the item}
    begin
      if top = 0 then stackempty;
      item := stack(top);
      top := top-1;
    end; {of delete}
```

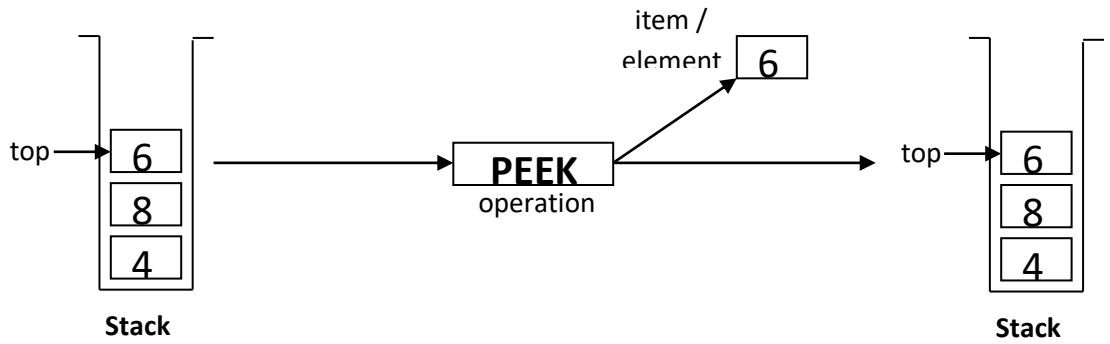


Implementation in C using array:

```
/* here, the variables stack, and top are global variables */
    int pop ( )
    {
        if (top == -1)
        {
            printf("Stack is Underflow");
            return (0);
        }
        else
        {
            return (stack[top--]);
        }}
```

III. Peek Operation:

- ✓ Returns the item at the top of the stack but does not delete it.
- ✓ This can also result in ***underflow*** if the stack is empty.



Algorithm:

PEEK(STACK, TOP)

```
BEGIN
    /* Check, Stack is empty? */
    if (TOP == -1) then
        print "Underflow" and return 0.
    else
        item = STACK[TOP]      / * stores the top element into a local variable */
        return item            / * returns the top element to the user */
END
```

Implementation in C using array:

```
/* here, the variables stack, and top are global variables */
    int pop ( )
    {
        if (top == -1)
        {
            printf("Stack is Underflow");
            return (0);
        }
        else
        {
            return (stack[top]);
        }}
```

Applications of Stack

1. It is very useful to evaluate arithmetic expressions. (Postfix Expressions)
2. Infix to Postfix Transformation
3. It is useful during the execution of recursive programs
4. A Stack is useful for designing the compiler in operating system to store local variables inside a function block.
5. A stack (memory stack) can be used in function calls including recursion.
6. Reversing Data
7. Reverse a List
8. Convert Decimal to Binary
9. Parsing – It is a logic that breaks into independent pieces for further processing
10. Backtracking

2. Describe the various applications of stack? (April 2011)

The various applications of a stack are

I. Arithmetic Expressions

II. Postfix evaluation

III. Infix to Postfix Conversion

IV. Function calls

I. Arithmetic Expressions

- An arithmetic expression will have operands and operators.
- Operator precedence listed below:

Highest	:	($)
Next Highest	:	(*) and (/)
Lowest	:	(+) and (-)

- For most common arithmetic operations, the operator symbol is placed in between its two operands. This is called ***infix notation***.

  *Example: A + B , E * F*

- Parentheses can be used to group the operations.

  *Example: (A + B) * C*

- Accordingly, the order of the operators and operands in an arithmetic expression does not uniquely determine the order in which the operations are to be performed.
- Polish notation refers to the notation in which the operator symbol is placed before its two operands. This is called ***prefix notation***.

  *Example: +AB, *EF*

- The fundamental property of polish notation is that the order in which the operations are to be performed is completely determined by the positions of the operators and operands in the expression.

- Accordingly, one never needs parentheses when writing expressions in Polish notation.
- ***Reverse Polish Notation*** refers to the analogous notation in which the operator symbol is placed after its two operands. This is called ***postfix notation***.

  *Example: AB+, EF**

- Here also the parentheses are not needed to determine the order of the operations.
- The computer usually evaluates an arithmetic expression written in infix notation in two steps,
  1. It converts the expression to **postfix notation**.
  2. It evaluates the postfix expression.
- In each step, the stack is the main tool that is used to accomplish the given task.

II. Postfix Evaluation

- Initialise an empty stack
- While token remain in the input stream
  - Read next token
  - If token is a number, push it into the stack
  - Else, if token is an operator, pop top two tokens off the stack, apply the operator, and push the answer back into the stack
- Pop the answer off the stack.

Algorithm postfix expression

```
Initialize a stack, opendstk to be empty.
{scan the input string reading one element at a time into symb }
While ( not end of input string )
        {
        Symb := next input character;
        If symb is an operand Then
                push (opndstk,symb)
        Else
                [symbol is an operator]
                {
                Opnd1:=pop(opndstk);
                Opnd2:=pop(opndnstk);
                Value := result of applying symb to opnd1 & opnd2
                Push(opndstk,value);
                }
Result := pop (opndstk);
```

Example:

6 2 3 + - 3 8 2 / + * 2 $ 3 +

Symbol	*Operand 1 (A)*	*Operand 2 (B)*	*Value (A ⊗B)*	*STACK*
6				6
2				6, 2
3				6, 2, 3
+	2	3	5	6, 5
-	6	5	1	1
3				1, 3
8				1, 3, 8
2				1, 3, 8, 2
/	8	2	/	1, 3, 4
+	3	4	7	1, 7
*	1	7	7	7
2				7, 2
$	7	2	49	49
3				49, 3
+	49	3	52	***52***

The Final value in the STACK is 52. This is the answer for the given expression.

III. Infix to Postfix Conversion

Infix expressions are often translated into postfix form in which the operators appear after their operands.

Steps:

1. Initialize an empty stack.
2. Scan the Infix Expression from left to right.
3. If the scannned character is an operand, add it to the Postfix Expression.
4. If the scanned character is an operator and if the stack is empty, then push the character to stack.
5. If the scanned character is an operator and the stack is not empty, Then
   (a) Compare the precedence of the character with the operator on the top of the stack.
   (b) While operator at top of stack has higher precedence over the scanned character & stack is not empty.
      (i) POP the stack.
      (ii) Add the Popped character to Postfix String.
   (c) Push the scanned character to stack.
6. Repeat the steps 3-5 till all the characters
7. While stack is not empty,
   (a) Add operator in top of stack

(b) Pop the stack.

8. Return the Postfix string.

Algorithm Infix to Postfix conversion (without parenthesis)

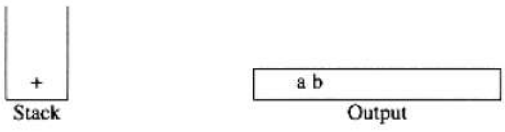
```
1. Opstk = the empty stack;
2. while ( not end of input )
   {
    symb = next input character;
3.  if ( symb is an operand )
            add symb to the Postfix String
4. else
    {
5. While( ! empty (opstk) && prec ( stacktop ( opstk), symb) )
    {
            topsymb = pop ( opstk )
            add topsymb to the Postfix String;
            }                                   / * end of while */
     Push(opstk, symb);
     }                                          /* end  else */
   }                                            /*  end while * /
6. While( ! empty ( opstk ) )
            {
            topsymb = pop (opstk)
            add topsymb to the Postfix String
            }                                   /*  end of while */
7. Return the Postfix String.
```

Example:

Suppose the infix expression is to be converted into postfix.

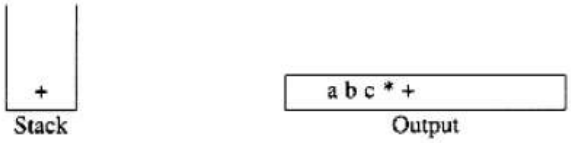
a + b * c + (d * e + f) * g

A correct answer is **a b c * + d e * f + g * +.**

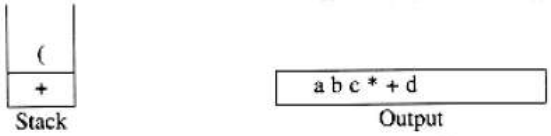


Next a '*' is read. The top entry on the operator stack has lower precedence than '*', so nothing is output and '*' is put on the stack. Next, *c* is read and output. Thus far,

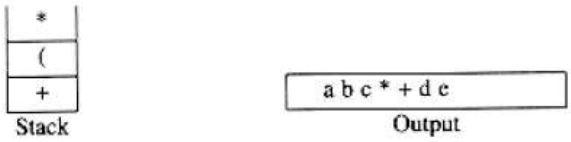
The next symbol is a '+'. Checking the stack, pop a '*' and place it on the output, pop the other '+', which is not of *lower* but equal priority, on the stack, and then push the '+'.



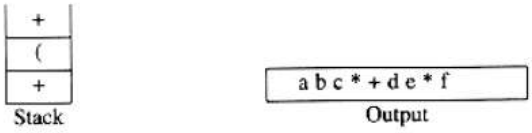
The next symbol read is an '(', which, being of highest precedence, is placed on the stack. Then *d* is read and output.



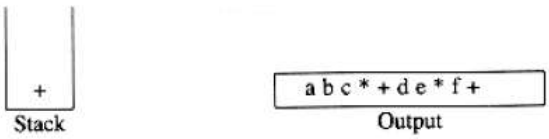
Continue by reading a '*'. Since open parentheses do not get removed except when a closed parenthesis is being processed, there is no output. Next, *e* is read and output.



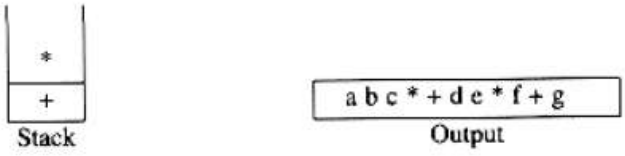
The next symbol read is a '+'. We pop and output '*' and then push '+'. Then read and output .



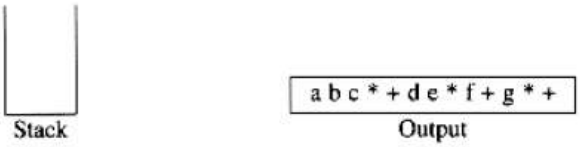
Now read a ')', so the stack is emptied back to the '('. Output a '+'.



Read a '*' next; it is pushed onto the stack. Then *g* is read and output.



The input is now empty, so we pop and output symbols from the stack until it is empty.



As before, this conversion requires only $O(n)$ time and works in one pass through the input. This algorithm does the right thing, because these operators associate from left to right.

IV. Function calls

push local data and return address onto stack

return by popping off local data and then popping off address and returning to it

return value can be pushed onto stack before returning, popped off by caller

3. Describe Queue and its operation with suitable example. (April 2012)

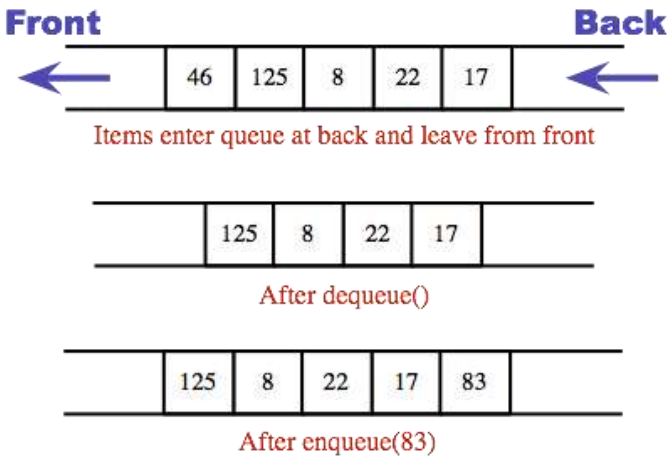
QUEUE

A queue is an ordered collection of items from which items may be deleted at one end called the front and the items may be inserted at the other end called rear of the queue.

PRINCIPLE

The first element inserted into a queue is the first element to be removed. Queue is called First In First Out (FIFO) list.

BASIC OPERATIONS INVOLVED IN A QUEUE:



1.Create a queue

2.Check whether a queue is empty or full

3.Add an item at the rear end

4.Remove an item at the front end

5.Read the front of the queue

6.Print the entire queue

INSERTION OPERATION

An attempt to push an item onto a queue, when the queue is full, causes an overflow.

1. Check whether the queue is full before attempting to insert another element.

2. Increment the rear pointer & 3. Insert the element at the rear pointer of the queue.

ALGORITHM:

Rear – Rear end pointer, Q – Queue, N – Total number of elements & Item – The element to be inserted

1. if(Rear=N) [Overflow?]

Then Call QUEUE_FULL

Return

2. Rear<-Rear+1 [Increment rear pointer]

3. Q[Rear]<-Item [Insert element]

End INSERT

DELETION OPERATION:

An attempt to remove an element from the queue when the queue is empty causes an underflow.

Deletion operation involves:

1. Check whether the queue is empty.

2. Increment the front pointer.

3. Remove the element.

ALGORITHM:

Q – Queue , Front – Front end pointer , Rear – Rear end pointer & Item – The element to be deleted.

1.if(Front=Rear) [Underflow?]

Then Call QUEUE_EMPTY

2. Front<-Front+1 [Incrementation]

3. Item<-Q [Front] [Delete element]

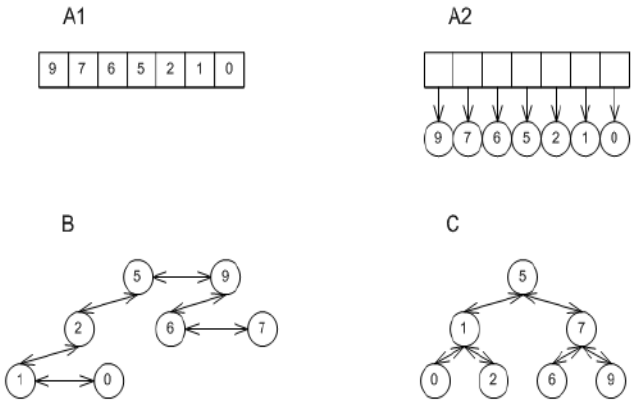
Thus queue is a dynamic structure that is constantly allowed to grow and shrink and thus changes its size, when implemented using linked list.

4. Write short notes on priority queues. (Nov 2012)

PRIORITY QUEUES

Priority queues are a kind of queue in which the elements are dequeued in priority order.

A priority queue is a collection of elements where each element has an associated priority. Elements are added and removed from the list in a manner such that the element with the highest (or lowest) priority is always the next to be removed. When using a heap mechanism, a priority queue is quite efficient for this type of operation.



- Each element has a **priority**, an element of a totally ordered set (usually a number).
- More important things come out *first*, even if they were added later.
- Three types of priority. Low priority [10], Normal Priority [5] and High Priority [1].
- There is no (fast) operation to find out whether an arbitrary element is in the queue.

ALGORITHM:

Priority Queue - Algorithms - Adjust

```
Adjust(i)
        left = 2i, right = 2i + 1

        if left <= H.size and H[left] > H[i]
                then largest = left
                else largest = i
        if right <= H.size and H[right] > H[largest]
                then largest = right
        if largest != i
                then swap H[largest] with H[i]
                        Adjust(largest)
```

Explanation

Adjust works recursively to guarantee the heap property. It compares the current node with its children finding which, if either, has a greater priority. If one of them does, it will swap array locations with the largest child. Adjust is then run again on the current node in its new location.

Priority Queue - Algorithms - Insert

```
Insert(Key)
        H.size = H.size + 1
        i = H.size
        while i > 1 and H[i/2] < Key
                H[i] = H[i/2]
                i = i/2
        end while
        H[i] = key
```

Explanation

The insert algorithm works by first inserting the new element (key) at the end of the array. This element is then compared with its parent for highest priority. If the parent has a lower priority, the two elements are swapped. This process is repeated until the new element has found its place.

Priority Queue - Algorithms - Extract

```
Extract()
        Max = H[1]
        H[1] = H[H.size]
        H.size = H.size - 1
        Adjust(1)
        Return Max
```

Explanation

Extract works by removing and returning the first array element, the one of highest priority, and then promoting the last array element to the first. Adjust is then run on the first element so that the heap property is maintained.

Run Time Complexity

- $\Theta(\lg n)$ time for Insert and worst case for Extract (where n is number of elements)
- $\Theta(\lg n)$ average time for Insert
- Can construct a Heap from a list in $\Theta(\lg n)$ where a Binary Search Tree takes $\Theta(n \lg n)$

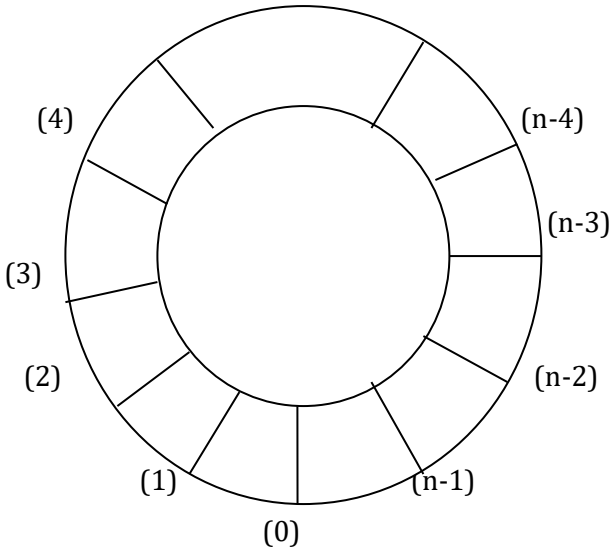
Space Requirements

- All operations are done in place - space requirement at any given time is the number of elements, n.

5. Explain circular queue with an example.

CIRCULAR QUEUE

- Initially in linear queue, the front and rear ends are in same position
  - (i.e.,-1).
- While inserting elements, the rear pointer moves one by one, until the last index position is reached.
- Beyond this the element cannot be inserted irrespective of the position of the front pointer.
- When deleting the elements, the front pointer moves one by one until the rear point is reached.
- If the front pointer reaches the rear pointer, both their positions are initialized to -1, and the queue is said to be empty.
- This is the main disadvantage of the linear queues, which is overcome in the circular queues.
- **Circular queue** is another form of a linear queue in which the last position is connected to the first position of the list.
- The circular queue is similar to linear queue has two ends, the front end and the rear end.
- The rear end is where we insert the elements and the front end is where we delete the elements.
- The traversing is in only one direction (i.e., from front to rear).
- Initially the front and rear ends are at same position(i.e., -1);
- While inserting elements the rear pointer moves one by one (where front pointer doesn't change) until the front end is reached.
- If the next position of the rear is front, the queue is said to be fully occupied.
- Beyond this insertion is not done.
- But if we delete any data, we can insert the element accordingly.
- While deleting the elements the front pointer moves one by one (where as the rear point doesn't change) until the rear point is reached.
- If the front pointer reaches the rear pointer, both the ir positions are initialized to -1, and the queue is said to be empty.
- A more efficient queue representation is obtained by the circular array.
- It becomes more convenient to declare the array as Q[n-1].
- When rear = n-1, the next element is entered at Q[0] in case that position is free.
- Using the same conventions, front will always point one position counterclockwise from the first element in the queue.
- Again, *front=rear* if and only if the queue is empty. Initially we have *front = rear* = 1.
- The following figure illustrates some of the possible configurations for a circular queue containing the four elements with n>4.

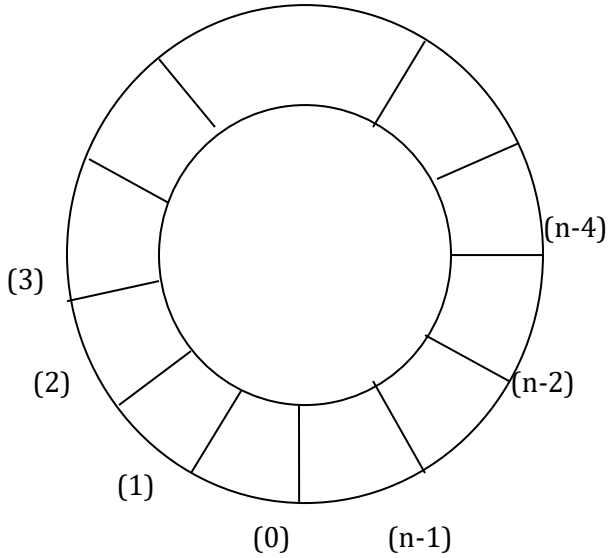


- *front = 0 ;rear=4*

- The assumption of circularity changes the ADD and DELETE algorithm slightly.
- In order to add an element , it will be necessary to move *rear* one position clockwise, i.e.,

  if *rear* = n-1 **then** *rear* =0

  else *rear = rear*+1.
- Using modulo operator which computes remainders, this is just *rear*=(*rear*+1)**mod n.**
- Similarly, it will be necessary to move **front** one position clockwise each time a deletion is made.
- Again, using the modulo operation, this can be accomplished by *front=(front*+1*)***mod** *n.*
- An examination of the algorithms indicates that addition and deletion can now be carried out in a fixed amount of time or ***O*(1).**



- **front = n-4 , rear = 0**

Procedure *ADDQ* (*item, Q, n, front, rear*)

//insert item in the circular queue stored in Q (0: n-1);

rear points to the last item and *front* is one position counterclockwise from the first item in Q//

rear = **(***rear*+1**)mod** *n* //advance rear clockwise//

if *front* = *rear* **then call** QUEUE-FULL

Q(*rear*)= item //insert new item //

end *ADDQ.*

Procedure *DELETEQ* (*item, Q, n, front, rear*)

//removes the front element of the queue Q (0: n-1)//

if *front* = *rear* **then call** QUEUE-EMPTY

front = **(***front* + 1**)mod** *n* //advance front clockwise//

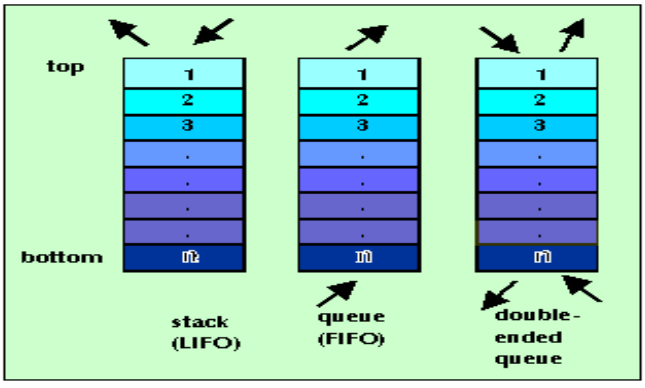
item = Q(*front*) //set item to front of queue//

end *DELETEQ*

6. Write an algorithm for a doubly ended queue for insertion and deletion

Double Ended Queue

A **deque** (short for *double-ended queue*) is an abstract data structure for which elements can be added to or removed from the front or back(both end). This differs from a normal queue, where elements can only be added to one end and removed from the other. Both queues and stacks can be considered specializations of deques, and can be implemented using deques.



double-ended queue

Two types of Dqueue are

1. Input Restricted Dqueue
2. Ouput Restricted Dqueue.

1. Input Restricted Dqueue

Where the input (insertion) is restricted to the rear end and the deletions has the options either end

2. Ouput Restricted Dqueue.

Where the output (deletion) is restricted to the front end and the insertions has the option either end

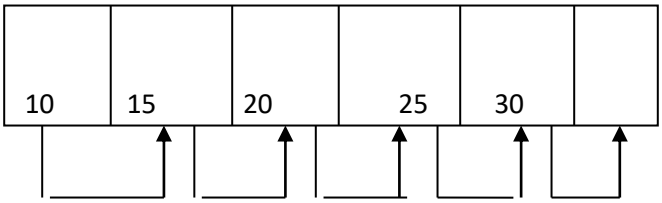
INSERT NEW ELEMENT INTO DEQUEUE:

AT THE END OF THE DEQUEUE:

- First check the dequeue is full or not.
- We have to check whether the element is first to be added or inserted if it is true assign front and rear is 0.
- Insert a new element

AT THE BEGINNING OF THE DEQUEUE:

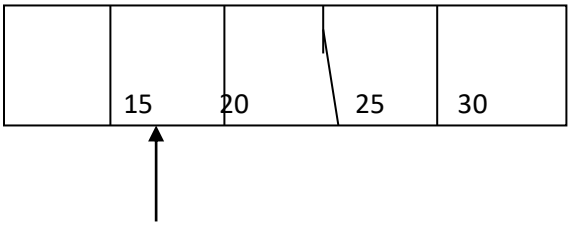
- First check the dequeue is full or not.
- We have to check whether the element is first to be inserted if it is true then perform the following steps
    1. shift the elements from left to right
    2. so that we need the number of elements present in the dequeue ,the position of the last element ie the position of the rear
    3. now insert any element in zeroth position



DELETING THE ELEMENT FROM DEQUEUE:

AT THE BEGINNING OF DEQUEUE:

- delete the first element from the front position of the dequeue
- the index of next element is stored in front pointer.
- Increment the front pointer by one .

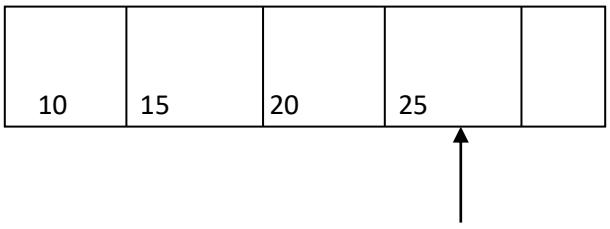


FRONT

AT THE END OF DEQUEUE:

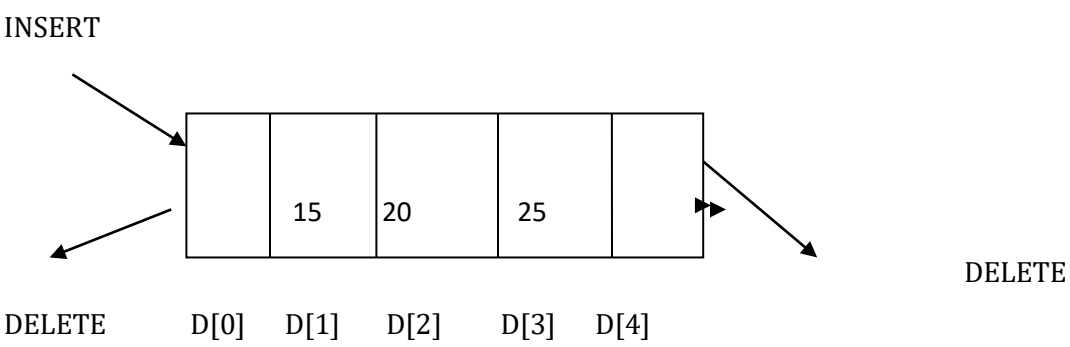
- The last element is deleted .
- The index of next element is stored in rear pointer

- Decrement the rear pointer by one.



INPUT RESTRICTED DEQUEUE:

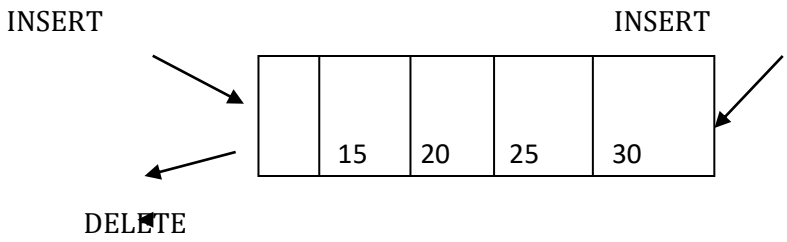
- It means we can insert the element only at one end and delete the elements at both ends.



- The above diagram shows the input restricted dequeue.

OUTPUT RESTRICTED DEQUEUE:

- It means we can insert the elements in both ends and delete the elements at only one end.



The above diagram shows the output restricted dequeue.

7. What is a linked list? What are its types?

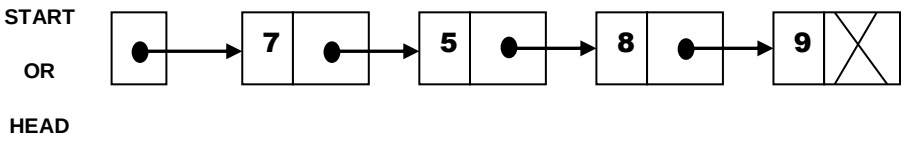
Linked List

- Some demerits of array, leads us to use linked list to store the list of items. They are,
  1. It is relatively expensive to insert and delete elements in an array.
  2. Array usually occupies a block of memory space, one cannot simply double or triple the size of an array when additional space is required. *(For this reason, arrays are called **"dense lists"** and are said to be **"static"** data structures.)*
- A ***linked list***, or ***one-way list***, is a linear collection of data elements, called ***nodes***, where the linear order is given by means of ***pointers***. That is, each node is divided into two parts:
  - ✓ The first part contains the information of the element i.e. INFO or DATA.

- ✓ The second part contains the ***link field***, which contains the address of the next node in the list.

- ❖ The linked list consists of series of nodes, which are not necessarily adjacent in memory.
- ❖ A list is a ***dynamic data structure*** i.e. the number of nodes on a list may vary dramatically as elements are inserted and removed.
- ❖ The pointer of the last node contains a special value, called the ***null*** pointer, which is any invalid address. This ***null pointer*** signals the end of list.
- ❖ The list with no nodes on it is called the ***empty list*** or ***null list***.

Example*: The linked list with 4 nodes.*

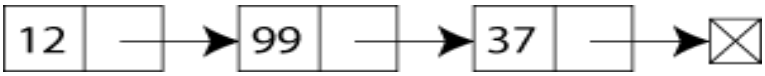


Types of Linked Lists:

a) Singly Linked List
b) Circular Linked List
c) Two-way or doubly linked lists
d) Circular doubly linked lists

Singly-linked list

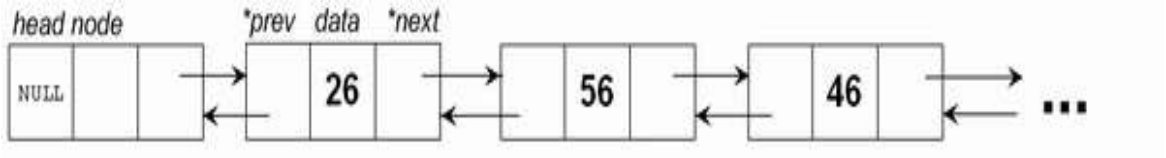
The simplest kind of linked list is a **singly-linked list** (or **slist** for short), which has one link per node. This link points to the next node in the list, or to a null value or empty list if it is the final node.



A singly linked list containing three integer values

Doubly-linked list

A more sophisticated kind of linked list is a **doubly-linked list** or **two-way linked list**. Each node has two links: one points to the previous node, or points to a null value or empty list if it is the first node; and one points to the next, or points to a null value or empty list if it is the final node.



An example of a doubly linked list.

Circularly-linked list

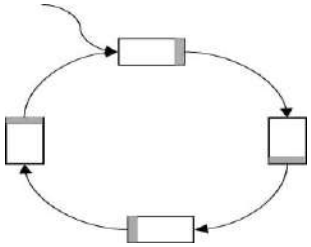
In a **circularly-linked list**, the first and final nodes are linked together. This can be done for both singly and doubly linked lists. To traverse a circular linked list, you begin at any node and follow the list in either direction until you return to the original node. Viewed another way, circularly-linked lists can be seen

as having no beginning or end. This type of list is most useful for managing buffers for data ingest, and in cases where you have one object in a list and wish to see all other objects in the list. The pointer pointing to the whole list is usually called the end pointer.

Singly-circularly-linked list

In a **singly-circularly-linked list**, each node has one link, similar to an ordinary *singly-linked list*, except that the next link of the last node points back to the first node.

As in a singly-linked list, new nodes can only be efficiently inserted after a node we already have a reference to. For this reason, it's usual to retain a reference to only the last element in a singly-circularly-linked list, as this allows quick insertion at the beginning, and also allows access to the first node through the last node's next pointer.



- Note that there is no **NULL** terminating pointer
- Choice of **head** node is arbitrary

Advantages of Linked List

1. Linked List is dynamic data structure; the size of a list can grow or shrink during the program execution. So, maximum size need not be known in advance.
2. The Linked List does not waste memory
3. It is not necessary to specify the size of the list, as in the case of arrays.
4. Linked List provides the flexibility in allowing the items to be rearranged.

8. Explain Single Linked List with its operations. (April 2013, Nov 2012)

SINGLY / LINEAR LINKED LIST:

In a sequential representation, suppose that the items were implicitly ordered, that is, each item contained within itself the address of the next item, such an implicit ordering gives rise to a data structure known as a Linear linked list or Singly linked list which is shown in figure:

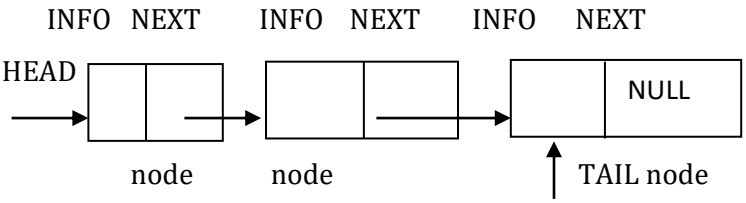


FIG: linear linked list.

Each node consists of two fieds,

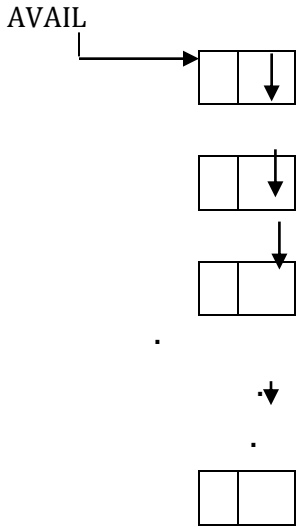
- information field called INFO that holds the actual element on the list and
- a pointer pointing to next element of the list called LINK ,ie. It holds the address of the next element.

The name of a typical element is denoted by NODE. Pictorially, the node structure is given as follows:

NODE

INFO	LINK

It is assumed that an available area of storage for this node structure consists of a stack of available nodes,as shown below:



Here, the pointer variable AVAIL contains the address of the top node in the stack. The head and tail are pointers pointing to first and last element of the list repectively. For an empty list the head and tail have the value NIL. When the list has one element, the head and tail point to the same.

OPERATIONS:

INSERTION IN A LIST:

Inserting a new item,say 'x' ,into the list has three situations:

1. Insertion at the front of the list
2. Insertion in the middle of the list or in the order
3. Insertion at the end of the list

INSERTION AT FRONT:

Algorithms:

1. Obtain space for new node
2. Assign data to the item field of new node
3. Set the next field of the new node to point to the start of the list
4. Change the head pointer to point to the new node.

Function INSERT(X, FIRST)

Variables used:

X ← New element to be inserted

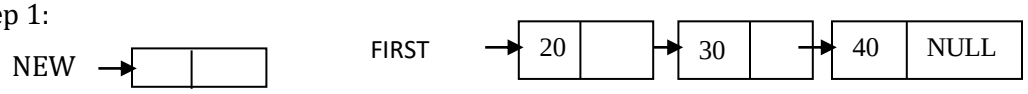
FIRST ← Pointer to the first element whose node contains
INFO and LINK fields.

AVAIL← Pointer to the top element of the availability stack

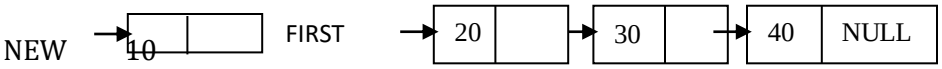
NEW ← Temporary pointer variable.

1. [Underflow?]
   If AVAIL = NULL
   Then Write('AVAILABILITY STACK UNDERFLOW')
   Return(FIRST)
2. [Obtain address of next free node]
   NEW ← AVAIL
3. [Remove free node from availability stack]
   AVAIL ← LINK(AVAIL)
4. [Initialize fields of new node and its link to the list]
   INFO(NEW) ← X
   LINK(NEW) ← FIRST
5. [Return address of new node]
   Return(NEW)

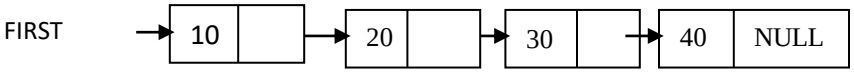
Step 1:



Step 2:



Step 3:



INSERTION IN THE MIDDLE OF THE LIST

Algorithms:

1. Set space for new node x
2. Assign value to the item field of x
3. Search for predecessor node n1 of x
4. Set the next field of x to point to link of node n1(node N2)
5. Set the next field of N1 to point to x.

Function INSORD(X, FIRST)

Variables used:

X← new element

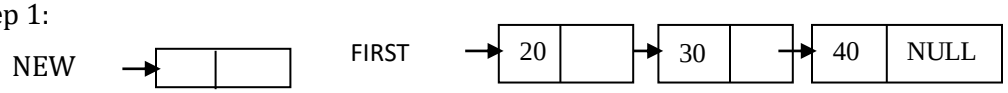
FIRST← Pointer to the first element whose node contains
INFO and LINK fields.

AVAIL← pointer to the top element of the availability stack

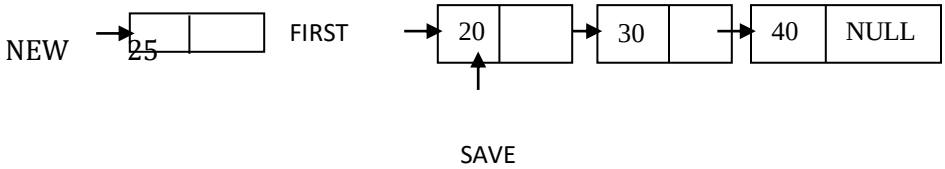
NEW,SAVE← Temporary pointer variables

1. [Underflow?]
   If AVAIL = NULL
   Then Write('AVAILABILITY STACK UNDERFLOW')
   Return(FIRST)
2. [Obtain address of next free node]
   NEW ← AVAIL
3. [Remove free node from availability stack]
   AVAIL ← LINK(AVAIL)
4. [Copy information contents into new node]
   INFO(NEW) ← X
5. [Is the list empty?]
   If FIRST = NULL
   Then LINK(NEW) ← FIRST
6. [Does the new node precede all others in the list?]
   If INFO(NEW) ≤ INFO(FIRST)
   Then LINK(NEW) ← FIRST
   Return(NEW)
7. [Initialize temporary pointer]
   SAVE ← FIRST
8. [Search for predecessor of new node]
   Repeat while LINK(SAVE) ≠ NULL and INFO(LINK(SAVE))≤ INFO(NEW)
   SAVE ← LINK(SAVE)
9. [Set link fields of new node and its predecessor]
   LINK(NEW) ← LINK(SAVE)
   LINK(SAVE)← NEW
10. [Return first node pointer]
    Return(FIRST)

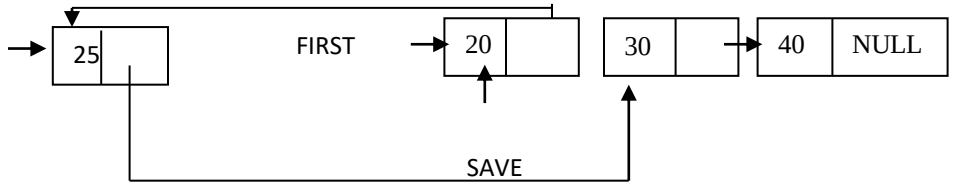
Step 1:



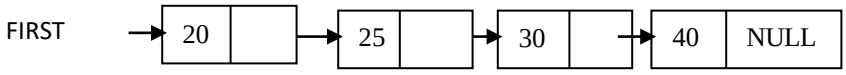
Step 2:



Step 3:



Step 4:



INSERTION AT THE END OF THE LIST

Algorithms:

1.Set space for new node x

2.Assign value to the item field of x

3.Set the next field of x to NULL

4.Set the next field of N2 to point to x

Function INSEND(X,FIRST)

Variables used:

X← new element

FIRST← Pointer to the first element whose node contains INFO and LINK fields.

AVAIL← pointer to the top element of the availability stack

NEW,SAVE← Temporary pointer variables

1. [Underflow?]
   If AVAIL = NULL
   Then Write('AVAILABILITY STACK UNDERFLOW')
   Return(FIRST)
2. [Obtain address of next free node]
   NEW ← AVAIL
3. [Remove free node from availability stack]
   AVAIL ← LINK(AVAIL)

4. [Initialize fields of new node]

   INFO(NEW) ← X

   LINK(NEW) ← NULL

5. [Is the list empty?]

   If FIRST = NULL

   Then Return(NEW)

6. [Initiate search for the last node]

   SAVE ← FIRST

7. [Search for end of list]

   Repeat while LINK(SAVE) ≠ NULL

   SAVE ← LINK(SAVE)

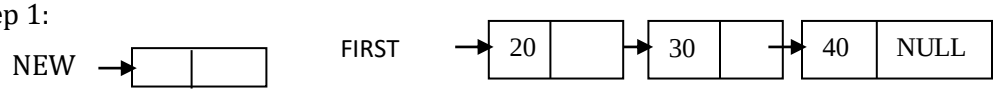
8. [Set LINK field of last node to NEW]

   LINK(SAVE) ← NEW

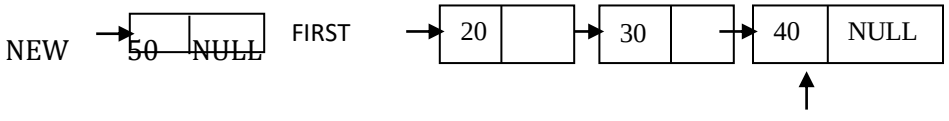
9. [Return first node pointer]

   Return(FIRST)

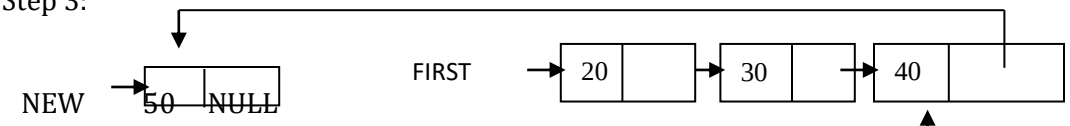
Step 1:



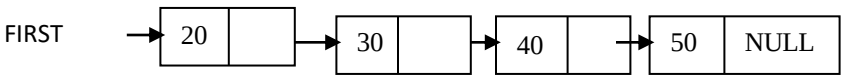
Step 2:



Step 3:



Step 4:



Note that no data is physically moved, as we had seen in array implementation. Only the pointers are readjusted.

DELETING AN ITEM FROM A LIST:

Deleting a node from the list requires only one pointer value to be changed, there we have situations:

1.Deleting the first item

2.Deleting the last item

3.Deleting between two nodes in the middle of the list.

Algorithms for deleting the first item:

1.If the element x to be deleted is at first store next field of x in some other variable y.

2.Free the space occupied by x

3.Change the head pointer to point to the address in y.

Algorithm for deleting the last item:

1.Set the next field of the node previous to the node x which is to be deleted as NULL

2.Free the space occupied by x

Algorithm for deleting x between two nodes N1 and N2 in the middle of the list:

1.Set the next field of the node N1 previous to x to point to the successor field N2 of the node x.

2.Free the space occupied by x.

Procedure DELETE(X, FIRST)

Variables used:

X ← New element to be inserted

FIRST ← Pointer to the first element whose node contains
INFO and LINK fields.

TEMP← To find the desired node

PRED← keeps track of the predecessor of TEMP

1. [Empty list?]
   If FIRST = NULL
   Then Write('UNDERFLOW")
   Return
2. [Initialize search for X]
   TEMP ← FIRST
3. [Find X]
   Repeat thru step 5 while TEMP ≠ X and LINK(TEMP) ≠ NULL
4. [Update predecessor marker]
   PRED ← TEMP
5. [Move to next node]
   TEMP ← LINK(TEMP)
6. [End of the list]
   If TEMP ≠ X
   Then Write('NODE NOT FOUND")
   Return
7. [Delete X]
   If X = FIRST (Is X the first node?)

Then FIRST ← LINK(FIRST)

Else LINK(PRED) ← LINK(X)

8. [Return node to availability area]

LINK(X) ← AVAIL

AVAIL ← X

Return

9. Explain Doubly Linked List with its operations. (April 2013)

DOUBLY LINKED LIST:

In the singly linked list ,we traverse the list in one dimension .In many application it is required to traverse a list in both direction .This 2 way traverse a list in both direction .This 2 way traverse can be realised by maintaining 2 link fields in each node instead of one .Each element of a doubly linked list structure has three fields

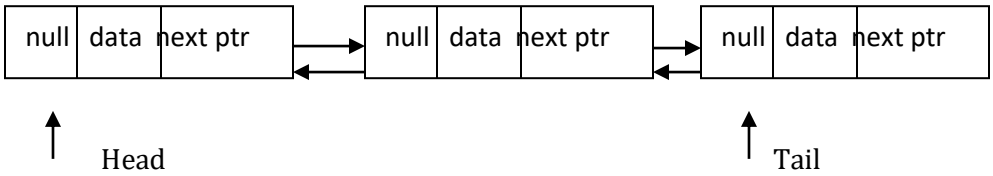
-data value

-a link to its successor

-a link to its predecessor

The predecessor link is called left link the successor link is known as right link

Thus the traversal of the list can be in any direction. such a structure is shown in following figure:

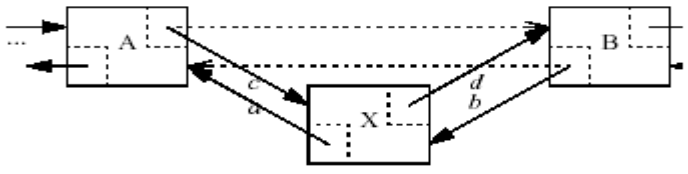


INSERTION OF A NODE:

To insert a node into a doubly linked list to the right of a specified node ,we have to consider several case ,these are as follow:

1. If the list is empty,insert a new node and make left and right link of new node to be set to nil.

2. If there is a predecessor and a successor to the given node .In such a case need to readjust link of the specified node and its successor node.



Insertion into a doubly linked list by getting new node and then changing pointers in order indicated

Procedure DOUBINS(L,R,M,X)

Variables used:

L ← left most node address

R ← right most node address

NEW← New node address to be inserted

LPTR← Left link of a node

RPTR← Right link of a node

INFO← Information field of the node

NODE← Name of an element of the list

M ← Pointer variable

X ← It contains information to be entered in the node

1. [Obtain new node from availability stack]

   NEW ← NODE

2. [Copy information field]

   INFO(NEW) ← X

3. [Insertion into an empty list?]

   If R = NULL

   Then LPTR(NEW) ← RPTR(NEW) ← NULL

   L ← R ← NEW

   Return

4. [Left most insertion?]

   If M = L

   Then LPTR(NEW) ← NULL

   RPTR(NEW) ← M

   LPTR(M) ← NEW

   L ← NEW

   Return

5. [Insert in middle]

   LPTR(NEW) ← LPTR(M)

   RPTR(NEW) ← M

   LPTR(M) ← NEW

   RPTR(LPTR(NEW)) ← NEW

   Return

3. If the insertion has to be done after the right most node in this list. In such a case only the right link of the specified node ie, the right most is to be changed.

DELETION OF A NODE:

Similar to insertion of a node ,we have to consider several case for deletion of a node.

Procedure DOUBDEL(L, R, OLD)

Variables used:

L← Left most node

R← Right most node

LPTR ← Left link of a node

RPTR ← Right link of a node

OLD ← The address of the node to be deleted

1. [Underflow?]

   If R = NULL

   Then Write('UNDERFLOW')

   Return

2. [Delete node]

   If L = R (Single node in list)

   Then L ← R← NULL

   Else IF OLD = L (Left most node being deleted)

   Then L ← RPTR(L)

   LPTR(L) ← NULL

   Else If OLD = R (Right most node being deleted)

   Then R ← LPTR(R)

   RPTR(R) ← NULL

   Else RPTR(LPTR(OLD)) ← RPTR(OLD)

   LPTR(RPTR(OLD)) ← LPTR(OLD)

3. [Return deleted node]

   Restore(OLD)

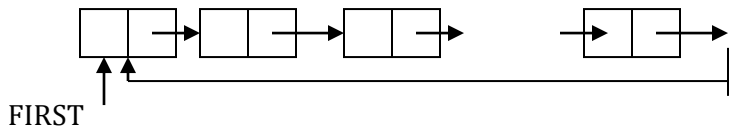
   Return

10. Explain about Circularly Linked List

CIRCULARLY LINKED LINEAR LIST:

In the singly linked linear list the last node consist of the NULL pointer. Slightly improvement in this type of linked list is accomplished by replacing the null pointer in the last node of a list with the address of its first node.such a list is called circularly linked linear list or simply a circular list

STRUCTURE OF A CIRCULAR LIST:



ADVANTAGE OF CIRCULAR LIST OVER SINGLY LINKED LISTS*:*

1. Accessibility of a node from a given node, every node is easily accessible i.e., all node be reached by merely chaining through the list

2. Deletion operation: In addition to the address of x the node to be deleted from a singly list, it is also necessary to give address of the first node of the list in order to the predecessor of X.

Such a requirement does not exist for a circular list, since the search for the predecessor of node X can be initiated from X itself.

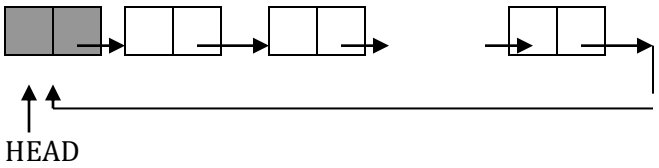
DISADVANTAGE OF A CIRCULAR LIST:

In processing a circular list, if we are not able to detect the end of the list, it is possible to get into an infinite loop.

This problem can be sowed i.e, the end of list can be detected by placing a special node which can be easily identified in the circular list .This special node is often called the list head of the circular list.

One more advantage of using this technique is that the list can never be empty which can be checked in the operation of singly linked list.

Representation of a circular list with a list head is shown below:



Here, variable HEAD denote the address of the list head .INFO field in the list head node is not used ,which is shown by the shading the field.

An empty list is represented by having LINK(HEAD)=HEAD.

The algorithm for inserting a node at the head of a circular list with a list head consist of the following steps:

NEW←NODE

INFO(NEW)←y

LINK(HEAD)←LINK(HEAD)

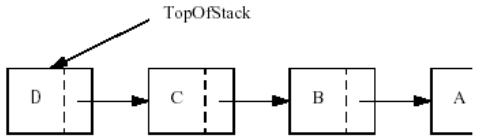
LINK(HEAD)←NEW

11. Describe about Linked Stacks and

LINKED STACKS:

- A *stack* is a list with the restriction that *inserts* and *deletes* can be performed in only one position, namely the end of the list called the *top*.
- The fundamental operations on a stack are *push*, which is equivalent to an insert, and *pop*, which deletes the most recently inserted element.
- The most recently inserted element can be examined prior to performing a *pop* by use of the *top* routine.
- A *pop* or *top* on an empty stack is generally considered an error in the stack ADT. On the other hand, running out of space when performing a *push* is an implementation error but not an ADT error.

- Stacks are sometimes known as LIFO (last in, first out) lists. The usual operations to make empty stacks and test for emptiness are part of the repertoire, but essentially all that you can do to a stack is *push* and *pop*.



Linked list implementation of the stack

Type declaration for linked list implementation of the stack ADT

```
typedef struct node *node_ptr;
struct node
{
element_type element;
ptrnode  next;
};
typedef ptrnode STACK;
```

Creating a stack

It allocates memory for the given structure (header of the stack. It points to the element at the front of the stack). The stack is emptied by calling make empty function.

```
STACK create_stack( void )
{
STACK S;
S = (STACK) malloc( sizeof( struct node ) );
if( S == NULL )
   fatal_error("Out of space!!!");
makeempty(S);
return S;
}
```

Routine to test whether a stack is empty

This function checks whether the stack is empty or not. The function is_empty () makes the S->next to point to the header in case if the stack is empty.

```
int is_empty( stack s )
{
return( S->next == NULL );
}
```

Routine to empty the stack

If the stack is not empty, the stack is popped until it becomes empty. In other words, the stack exists but the stack is empty.

```
void makeempty( STACK S )
{
    if( S == NULL )
      error("Must use create_stack first");
    else
      while(!Isempty(S))
          pop(S);
}
```

Routine to push onto a stack

To push an element into the stack, allocate memory, insert the value and make S->next to point to that element.

```
void push( element_type x, STACK S )
{
   Ptrnode tmpcell;
   tmpcell = (node_ptr) malloc( sizeof ( struct node ) );
    if( tmpcell == NULL )
         fatal_error("Out of space!!!");
else
{
tmpcell->element = x;
tmpcell->next = S->next;
S->next = tmpcell;
}
}
```

Routine to return the top element in a stack

This function returns the element in the top of the stack.

```
element_type top( STACK S )
{
if( !isempty( S ) )
   return S->next->element;
error("Empty stack");
return 0;
}
```

Routine to pop from a stack

Check whether the stack is empty. Declare a pointer variable firstcell make it point to the element that is to be deleted. If the stack is not empty the element pointed to by S->next should be removed from the stack.

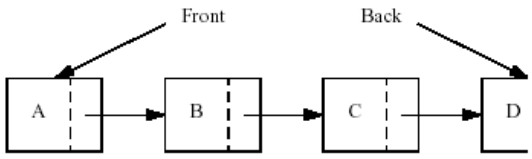
```
void pop( STACK S )
{
Ptrnode  first_cell;
 if( isempty( S ) )
    error("Empty stack");
 else
  {
        first_cell = S->next;
        S->next = S->next->next;
        free( first_cell );
  }
}
```

11. Describe about Linked Queues

Linked Queues

Like stacks, queues are lists. With a queue, however, insertion is done at one end, whereas deletion is performed at the other end. The basic operations on a queue are enqueue, which inserts an element at the end of the list (called the rear), and dequeue, which deletes (and returns) the element at the start of the list (known as the front).

- Figure shows the abstract model of a queue. The queues can be implemented using arrays or pointers.
- Queue is implemented using Singly Linked Lists, where the next pointer of the element at the rear end is made to point to NULL.



Linked list implementation of the queue

Queue Model

The basic operations on a queue are *enqueue,* which inserts an element at the end of the list (called the rear), and *dequeue,* which deletes (and returns) the element at the start of the list (known as the front). Figure shows the abstract model of a queue.

Structure definition

Each node contains two fields. Every last node points to null.

```
struct node
{
        element type element ;
        ptr to node*next;
};
typedef node_ptr queue;
```

Function to check whether the queue is empty or not

This function is empty checks whether the queue is empty or not, It returns true, if the queue is empty. It makes the q->next to point to NULL in case if the queue is empty.

```
int is_Empty (queue q)
{
    return q->next= = NULL;
}
```

Creating a Queue

It allocates memory for the given structure (header of the queue. It points to the element at the front of the queue). The queue is emptied by calling makeempty( ) function.

```
queue create queue (void)
{
queue q;
q=malloc (sizeof(struct node));
if(q= = null)
    fatal error("OUT OF SPACE"):
makeempty(q);
return q;
}
```

Function to empty the queue

If the queue is not empty, the elements in the queue are removed until the queue becomes empty. In other words, the queue exists but the queue is empty.

```
void make empty(queue q)
{
if(q = =null)
  error("Must use create queue first");
else
while (!isempty (q));
  delete(q);
}
```

Function to insert an element into the queue

To insert an element into the queue, in the linked list implementation, it is possible only at the rear end. Therefore the position of the last element (p) is found. The last element's next pointer points to NULL.

```
void insert (elementtype X, queue q)
{
ptr tonode tempcell;
p=q;
tempcell=malloc(sizeof (struct node));
if(tempcell==NULL)
 fatalerror("out of space");
else
{
while (p->next!=NULL)
    p=p->next;
tempcell->element=X;
tempcell->next=p->next;
p->next=tempcell;
}
}
```

Function to return the first element in the list

It checks whether the queue is empty. If it is not empty, it returns the element at the front of the queue.

```
int front(queue q)
{
if(!isempty (q))
  return q->next->element;
else
  fatal error("empty queue");
return 0;
}
```

Function to remove an element at the front of the queue (first)

Check whether the queue is empty. If the queue is not empty the element pointed to by q-> next should be removed from the queue. Declare a pointer variable firstcell. Make it point to the element that is to be deleted. Once the first element is removed, q->next should point to the next element in the queue.

```
void delete(queue q)
{
ptr to node firstcell;
```

```
if (isempty (q))
    error("empty queue");
else
{
firstcell=q->next;
q->next=q->next->next;
free(first cell);
}
}
```

13. Explain any of the application of Linked List with example. (Nov 11, April 2013, Nov 2012)

APPLICATION OF LINKED LIST:

The Polynomial ADT

- An abstract data type for single-variable polynomials (with nonnegative exponents) can be defined by using a list.
- If most of the coefficients a_i are nonzero, use a simple array to store the coefficients. Routines can be written to perform addition, subtraction, multiplication, differentiation, and other operations on these polynomials. In this case, use the type declarations below.
- Two possibilities are addition and multiplication.
- Ignoring the time to initialize the output polynomials to zero, the running time of the multiplication routine is proportional to the product of the degree of the two input polynomials. This is adequate for dense polynomials, where most of the terms are present, but if $p_1(x) = 10x^{1000} + 5x^{14} + 1$ and $p_2(x) = 3x^{1990} - 2x^{1492} + 11x + 5$, then the running time is likely to be unacceptable.
- Most of the time is spent multiplying zeros and stepping through what amounts to nonexistent parts of the input polynomials. This is always undesirable.

```
typedef struct
{
int coeff_array[ MAX_DEGREE+1 ];
unsigned int high_power;
} *POLYNOMIAL;
```

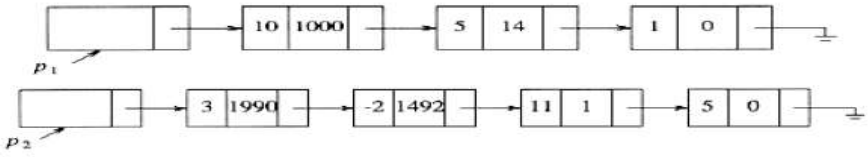
Type declarations for array implementation of the polynomial ADT

- An alternative is to use a singly linked list. Each term in the polynomial is contained in one cell, and the cells are sorted in decreasing order of exponents. For instance, the linked lists in represent $p_1(x)$ and $p_2(x)$.

```
void zero_polynomial( POLYNOMIAL poly )
{
    unsigned int i;
```

```
        for( i=0; i<=MAX_DEGREE; i++ )
        poly->coeff_array[i] = 0;
        poly->high_power = 0;
    }
```

Procedure to initialize a polynomial to zero



Linked List representation of 2 polynomials

```
void add_polynomial( POLYNOMIAL poly1, POLYNOMIAL poly2,
                     POLYNOMIAL poly_sum )
{
    int i;
    zero_polynomial( poly_sum );
    poly_sum->high_power = max( poly1->high_power,
    poly2->high_power);
    for( i=poly_sum->high_power; i>=0; i-- )
    poly_sum->coeff_array[i] = poly1->coeff_array[i] + poly2->coeff_array[i];
}
```

Procedure to add two polynomials

```
void mult_polynomial( POLYNOMIAL poly1, POLYNOMIAL poly2, POLYNOMIAL
                      poly_prod )
{
unsigned int i, j;
zero_polynomial( poly_prod );
poly_prod->high_power = poly1->high_power + poly2->high_power;
if( poly_prod->high_power > MAX_DEGREE )
   error("Exceeded array size");
else
        for( i=0; i<=poly->high_power; i++ )
for( j=0; j<=poly2->high_power; j++ )
    poly_prod->coeff_array[i+j] += poly1->coeff_array[i] * poly2->coeff_array[j];
}
```

Procedure to multiply two polynomials

```
typedef struct node *node_ptr;
struct node
{
int coefficient;
int exponent;
node_ptr next;
} ;
typedef node_ptr POLYNOMIAL; /* keep nodes sorted by exponent */
```

Type declaration for linked list implementation of the Polynomial ADT

The operations would then be straightforward to implement. The only potential difficulty is that when two polynomials are multiplied, the resultant polynomial will have to have like terms combined.

Radix Sort

- A second example where linked lists are used is called *radix sort.* Radix sort is sometimes known as *card sort,* because it was used, until the advent of modern computers, to sort old-style punch cards.
- If there are *n* integers in the range 1 to *m* (or 0 to *m - 1)* 9, this information can be used to obtain a fast sort known as *bucket sort*.
- Keep an array called *count,* of size *m,* which is initialized to zero. Thus, *count* has *m* cells (or buckets), which are initially empty. When a_i is read, increment (by one) *count*[a_i]. After all the input is read, scan the *count* array, printing out a representation of the sorted list. This algorithm takes $O(m + n)$; the proof is left as an exercise. If $m = O(n)$*, then bucket sort is* $O(n)$*.*
- Radix sort is a generalization of this. The easiest way to see what happens is by example. Suppose there are 10 numbers, in the range 0 to 999, that are to be sorted. In general, this is *n* numbers in the range 0 to $n^p - 1$ for some constant *p.*
- Obviously, bucket sort cannot be used; there would be too many buckets. The trick is to use several passes of bucket sort.
- The natural algorithm would be to bucket-sort by the most significant "digit" (digit is taken to base *n*), then next most significant, and so on. That algorithm does not work, but if bucket sorts are performed by least significant "digit" first, then the algorithm works. Of course, more than one number could fall into the same bucket, and, unlike the original bucket sort, these numbers could be different, so keep them in a list.
- The following example shows the action of radix sort on 10 numbers. The input is 64, 8, 216, 512, 27, 729, 0, 1, 343, 125 (the first ten cubes arranged randomly). The first step bucket sorts by the least significant digit. In this case the math is in base 10 (to make things simple), but do not assume this in general. The buckets are as shown in Figure, so the list, sorted by least significant digit, is 0, 1, 512, 343, 64, 125, 216, 27, 8, 729.
- These are now sorted by the next least significant digit (the tens digit here). Pass 2 gives output 0, 1, 8, 512, 216, 125, 27, 729, 343, 64. This list is now sorted with respect to the two least significant digits.

The final pass, bucket-sorts is done by most significant digit. The final list is 0, 1, 8, 27, 64, 125, 216, 343, 512, 729.

- To see that the algorithm works, notice that the only possible failure would occur if two numbers came out of the same bucket in the wrong order. But the previous passes ensure that when several numbers enter a bucket, they enter in sorted order.
- The running time is O(p(n + b)) where p is the number of passes, n is the number of elements to sort, and b is the number of buckets. In our case, b = n.

0 1 512 343 64 125 216 27 8 729

---

0 1 2 3 4 5 6 7 8 9

Buckets after first step of radix sort

8 729

1 216 27

0 512 125 343 64

---

0 1 2 3 4 5 6 7 8 9

Buckets after the second pass of radix sort

64

27

8

1

0 125 216 343 512 729

---

0 1 2 3 4 5 6 7 8 9

Buckets after the last pass of radix sort